"SyncLock: Concurrent File Access Management System"

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

Ch.Yaswanth **2520030259**

V.Bharath Kumar **2520030169**

Praneeth **2520030186**

Academic Year 2026–2027

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Hyderabad, Telangana, India

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

DECLARATION

The Project Report entitled "SyncLock: Concurrent File Access Management System" is a record of bonafide work carried out and submitted by Ch.Yaswanth – 2520030259, V.Bharath Kumar – 2520030169, and Praneeth – 2520030186 in partial fulfillment for the award of the degree of B.Tech in Computer Science Engineering at K L (Deemed to be) University. The work presented in this report is based on the team's implementation, testing, and demonstration of a Linux-based concurrent file access management system. The results embodied in this report have not been copied from any other department, university, or institute.

Ch.Yaswanth	**2520030259**
V.Bharath Kumar	**2520030169**
Praneeth	**2520030186**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the project report entitled “SyncLock: Concurrent File Access Management System” is a bonafide work done and submitted by Ch.Yaswanth – 2520030259, V.Bharath Kumar – 2520030169, and Praneeth – 2520030186, in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of Computer Science Engineering, K L (Deemed to be) University, during the academic year 2026–2027.

Signature of the Supervisor

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the support, encouragement, and assistance of many people. We sincerely thank everyone who contributed directly or indirectly to the development and completion of SyncLock.

We express our gratitude to the Department of Computer Science Engineering and the faculty members of K L (Deemed to be) University for providing the academic environment, infrastructure, and resources required to carry out this project.

We are thankful to the teaching and non-teaching staff members who supported us during the development, testing, documentation, and demonstration of the project. Their suggestions and technical guidance helped us improve the quality of the implementation.

Finally, we thank our parents, friends, classmates, and everyone who encouraged us throughout the project. Their cooperation and motivation were valuable in completing SyncLock successfully.

Ch.Yaswanth 2520030259

V.Bharath Kumar 2520030169

Praneeth 2520030186

ABSTRACT

Modern operating systems frequently execute multiple processes that may access the same file concurrently. Without coordination, simultaneous updates can produce race conditions, lost updates, inconsistent data, and file corruption. SyncLock is a Linux-based Concurrent File Access Management System designed to control shared file access and demonstrate practical operating-system synchronization. The system uses Python's fcntl.flock() interface to implement shared read locks and exclusive write locks. Multiple processes can read the same file when no incompatible writer is active, while a write operation requires exclusive access. File versioning is used as an additional consistency mechanism: an update is accepted only when the client is working with the current version. Safe update handling is supported through temporary-file writing followed by atomic replacement. The system records file metadata, operations, versions, and conflict events in SQLite. A FastAPI backend exposes the file-management and monitoring functionality, while an HTML, CSS, and JavaScript dashboard presents system statistics and activity. A multiprocessing simulator creates real concurrent processes to demonstrate contention for the same shared resource. The project therefore connects operating-system concepts such as mutual exclusion, critical sections, reader-writer synchronization, process concurrency, and file consistency with a practical web-based demonstration.

Team Name: SyncLock

Team Members:

1. Ch.Yaswanth – 2520030259
2. V.Bharath Kumar – 2520030169
3. Praneeth – 2520030186

INDEX

S. No.	Chapter / Topic	Page No.
	Acknowledgement	i
	Abstract	ii
1	Introduction	7
1.1	Background of the Project	1
1.2	Problem Statement	2
1.3	Scope of the Project	2
1.4	Objectives of the Project	3
1.5	Importance of the Application	3
1.6	Target Users / Audience	4
2	Literature Survey	9
3	Problem Statement and Objectives	11
4	Hardware and Software Requirements	12
5	Technology and Tools Used	14
6	System Architecture and Design	16
7	Implementation	19
8	Execution, Results and Discussion	21
9	Conclusion and Future Scope	23
10	References	25

CHAPTER - 1
INTRODUCTION

1.1 Background of the Project

File access is a fundamental operating-system operation, but concurrent access introduces synchronization challenges. When two or more independent processes operate on the same file, the final state depends on the ordering and timing of their operations. A process may read an older version while another process is already updating the file. If both subsequently write their results, the earlier update can be lost. Such race conditions are especially important when the shared file represents configuration, records, logs, or other data that must remain consistent.

SyncLock was developed as a practical solution to this problem. The project treats a shared file as a protected resource and uses Linux advisory file locking to coordinate access between processes. The design distinguishes between read and write operations, allowing compatible readers to proceed while ensuring that a writer receives exclusive access. The system also maintains a logical version number for every managed file so that stale updates can be identified before they replace newer data.

1.2 Problem Statement

To develop a concurrent file access management system that prevents race conditions, conflicting writes, lost updates, and data inconsistency when multiple processes access and modify a shared file simultaneously.

1.3 Scope of the Project

- Manage a controlled collection of shared text files.
- Provide safe read and write operations through a FastAPI backend.
- Use Linux file locking to coordinate concurrent processes.
- Support shared read access and exclusive write access.
- Detect stale writes using file version numbers.
- Perform safe updates using temporary files and atomic replacement.
- Record operations and conflicts for monitoring and demonstration.
- Provide a browser-based dashboard showing file and synchronization status.
- Provide a multiprocessing simulator for live hackathon demonstration.

1.4 Objectives of the Project

- Prevent simultaneous incompatible writes to the same shared file.
- Allow multiple compatible readers to access a file concurrently.
- Demonstrate mutual exclusion and critical-section control using Linux.
- Detect stale client updates using version validation.
- Maintain a reliable audit trail of operations and conflicts.
- Provide a simple and understandable interface for judges and users.
- Demonstrate real process-level concurrency rather than only simulated UI events.

1.5 Importance of the Application

The project converts a core operating-system concept into a visible, testable application. Instead of showing synchronization only as a theoretical topic, SyncLock allows users to observe several processes competing for the same resource and see how locks determine which operations can proceed. This makes the project suitable for an academic demonstration and for a hackathon where the working mechanism must be explained clearly.

1.6 Target Users / Audience

- Students learning operating systems, process synchronization, and concurrency.
- Developers building multi-process applications that access shared files.
- Faculty and evaluators who want a visual demonstration of OS synchronization.
- Hackathon judges evaluating a practical implementation of race-condition prevention.

CHAPTER - 2
LITERATURE SURVEY

The literature survey focuses on existing approaches to file locking, shared-resource coordination, and concurrency. The reviewed systems show that synchronization can be implemented at different layers, from operating-system file locks to application-level and distributed lock managers. SyncLock uses the local Linux file-system approach because the hackathon problem is specifically centered on concurrent access to a shared file.

Linux flock / flock(2)

Linux provides advisory file locking through flock(2). Shared locks allow more than one process to hold a read-style lock, while an exclusive lock allows only one process at a time. SyncLock directly applies this principle to managed files.

Reference: https://man7.org/linux/man-pages/man2/flock.2.html

Python fcntl.flock()

Python exposes the operating-system locking mechanism through the fcntl module. The flock() function accepts shared, exclusive, unlock, and non-blocking operations, making it suitable for implementing the lock manager in SyncLock.

Reference: https://docs.python.org/3/library/fcntl.html

SQLite Locking and Concurrency

SQLite is an embedded database that uses locking and transaction mechanisms to maintain consistency. Its documentation describes concurrency, isolation, and locking behavior. SyncLock uses SQLite for metadata and audit logging rather than as the lock itself.

Reference: https://sqlite.org/docs.html

Git LFS File Locking

Git LFS provides a file-locking model for collaborative editing, particularly for files that cannot be merged easily. The idea of explicitly claiming exclusive edit access is related to SyncLock, although SyncLock works directly with Linux file locks and local processes.

Reference: https://github.com/git-lfs/git-lfs/blob/main/docs/man/git-lfs-lock.adoc

GitLab File Locking

GitLab supports exclusive file locks to prevent multiple collaborators from editing the same file simultaneously. This demonstrates the practical value of file-level coordination in collaborative environments.

Reference: https://docs.gitlab.com/user/project/file_lock/

Redis Distributed Locks

Redis documents distributed lock patterns for systems where different processes or machines must coordinate access to shared resources. SyncLock is intentionally simpler and local, but the comparison shows how the same mutual-exclusion principle can be extended to distributed systems.

Reference: https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/

Survey Summary

Approach	Lock Scope	Main Strength	Limitation for This Project
Linux flock	Local file/process	Simple OS-level coordination	Local filesystem scope
Python fcntl.flock	Local file/process	Direct Python access to OS locking	Unix/Linux oriented
SQLite	Database file	Transactions and consistency	Not a general-purpose file lock manager
Git LFS locking	Repository file	Collaborative exclusive editing	Requires Git/LFS workflow
GitLab file locking	Project files	Team-level visibility	Platform-specific
Redis locks	Distributed resources	Can coordinate across machines	More infrastructure and failure cases

CHAPTER - 3
PROBLEM STATEMENT AND OBJECTIVES

3.1 Problem Statement

Multiple independent processes may attempt to read or update a shared file at overlapping times. If these operations are not coordinated, a process can overwrite changes made by another process or leave the file in an unexpected state. The project therefore requires a mechanism that enforces compatible concurrent access while protecting the integrity of updates.

3.2 Problem Description

The core challenge is to coordinate concurrent file operations without allowing incompatible read/write or write/write operations to modify the shared resource at the same time. The solution must also detect stale data and provide evidence of how the system handled contention.

3.3 Proposed Solution

SyncLock introduces a lock-controlled file access layer. Before a process performs a read, it requests a shared lock. Before a process modifies a file, it requests an exclusive lock. A version number is checked before a write is committed, and the new content is written to a temporary file before replacing the original file. Every important operation is recorded in SQLite so that the dashboard can show successful operations and blocked conflicts.

3.4 Objectives

- Implement shared and exclusive locking.
- Prevent lost updates and inconsistent writes.
- Demonstrate process-level concurrency on Ubuntu.
- Track file versions and operation outcomes.
- Provide a web interface for monitoring and demonstration.
- Create a clear, reproducible hackathon workflow.

CHAPTER - 4
HARDWARE AND SOFTWARE REQUIREMENTS

4.1 Hardware Requirements

Component	Minimum / Recommended Requirement
Processor	Dual-core processor or better
RAM	4 GB minimum; 8 GB or more recommended
Storage	At least 1 GB free for source code, database, and test files
Display	Standard monitor capable of running a modern web browser
Network	Not required for local execution; optional for package installation

4.2 Software Requirements

Software	Purpose
Ubuntu Linux	Operating-system environment and native file locking
Python 3	Backend, lock manager, file operations, simulator
FastAPI	REST API and web backend
Uvicorn	ASGI server used to run FastAPI
SQLite	Persistent metadata and operation logging
HTML / CSS / JavaScript	Browser-based dashboard
Python multiprocessing	Creation of concurrent worker processes
Git	Version control and project management
Web Browser	Access to the SyncLock dashboard

4.3 Operating System Environment – Ubuntu

The project is designed to run in Ubuntu because the synchronization layer uses the Unix/Linux fcntl interface. The application can be executed from an Ubuntu terminal and accessed through a browser at the local server address. A Windows machine may also host Ubuntu through WSL2, but the locking mechanism and project execution are treated as Linux operations in this documentation.

4.4 Development Tools

- Visual Studio Code or another source-code editor.
- Ubuntu Terminal / Bash.
- Python virtual environment and pip.
- FastAPI and Uvicorn.
- SQLite command-line tools or Python's sqlite3 module.
- Browser developer tools for frontend debugging.

CHAPTER - 5
TECHNOLOGY AND TOOLS USED

5.1 Python

Python is the primary implementation language. It is used for file handling, operating-system interaction, locking, database access, API development, and process simulation. Python also provides the fcntl and multiprocessing modules required by the project.

5.2 FastAPI

FastAPI is used to build the backend API layer. It receives requests from the browser, validates inputs, invokes file-management operations, and returns structured responses to the dashboard.

5.3 Uvicorn

Uvicorn is used as the ASGI server for running the FastAPI application locally. During demonstration, the server is started on port 8000 and the dashboard is opened in a browser.

5.4 fcntl.flock()

The fcntl.flock() function is the core synchronization mechanism. LOCK_SH is used for shared read access, LOCK_EX for exclusive write access, and LOCK_UN to release a lock. LOCK_NB can be used when the application needs a non-blocking attempt that reports contention instead of waiting.

```
import fcntl

with open(file_path, "r") as file:
    fcntl.flock(file.fileno(), fcntl.LOCK_SH)
    data = file.read()
    fcntl.flock(file.fileno(), fcntl.LOCK_UN)
```

5.5 SQLite

SQLite stores persistent metadata such as managed file names, versions, operation records, and conflict events. The database is separate from the protected file content, which remains in the shared-files storage directory.

5.6 HTML, CSS and JavaScript

The dashboard uses HTML for structure, CSS for visual presentation, and JavaScript for API communication and dynamic updates. The interface displays shared files, operation counts, blocked conflicts, successful writes, and recent activity.

5.7 multiprocessing

Python multiprocessing is used to create independent worker processes. This is important because the demonstration must show actual concurrent processes contending for a shared resource rather than merely displaying pre-generated messages.

5.8 Supporting Utilities

- os and pathlib for safe filesystem operations.

- tempfile for temporary files used during safe writes.
- time and datetime for timestamps and operation duration.
- sqlite3 for database connectivity.
- JSON/HTTP handling through FastAPI responses.
- Bash/Linux commands for execution and process observation.

CHAPTER - 6
SYSTEM ARCHITECTURE AND DESIGN

6.1 Overall System Architecture

The architecture separates the browser interface from the synchronization logic. The browser communicates with the FastAPI backend. The backend validates the requested operation, invokes the file manager and lock manager, updates metadata in SQLite, and returns the result to the dashboard.

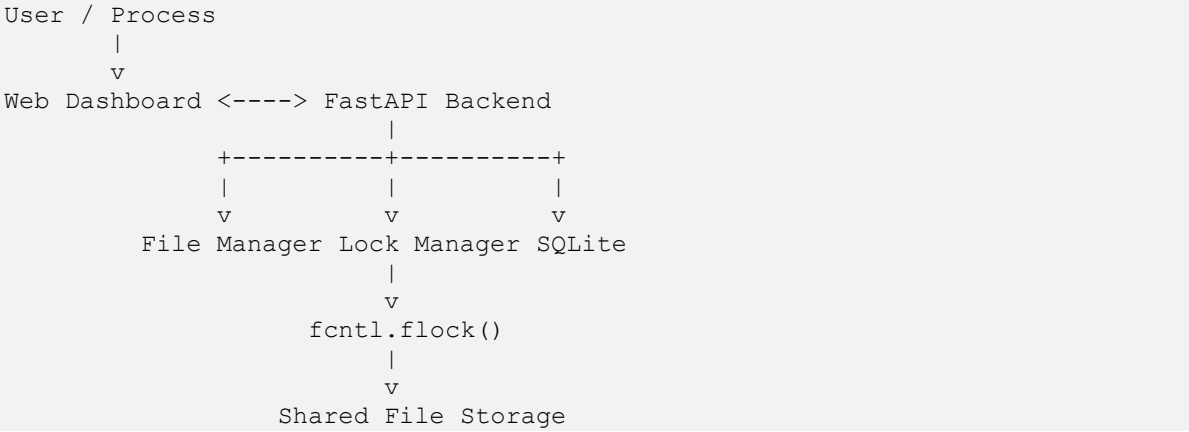


Figure 6.1: High-level architecture of SyncLock

6.2 System Workflow

1. The user creates or selects a shared file from the dashboard.
2. The backend verifies that the file name is valid and belongs to the managed storage area.
3. A read or write operation is requested.
4. The lock manager attempts to obtain a compatible Linux file lock.
5. If the lock is available, the operation proceeds; otherwise the request is blocked or reported as a conflict depending on the operation mode.
6. For writes, the expected file version is checked against the current version.
7. The new content is written safely and the file version is incremented.
8. The lock is released.
9. The operation result is logged in SQLite and the dashboard statistics are refreshed.

6.3 Flowchart / Data Flow



```
  |                                                        |
  v                                                        |
Perform Operation <----------------------------------------+
  |
  v
For WRITE: Check Version
  |
  +---- stale version? ---- YES ----> Reject + Log Conflict
  |
  NO
  |
  v
Atomic Update + Version Increment
  |
  v
Release Lock
  |
  v
Log Operation
  |
  v
Refresh Dashboard
  |
  v
END
```

Figure 6.2: SyncLock operation flow

6.4 Module Description

Module	Responsibilities
File Management	Create, read, update, delete, and validate shared files.
Lock Manager	Acquire/release shared and exclusive Linux file locks.
Version Manager	Track file versions and reject stale updates.
Atomic Writer	Write to a temporary file and replace the target safely.
Database Logger	Store file metadata, operations, and conflict information.
API Layer	Expose backend functionality to the browser.
Dashboard	Display files, statistics, lock/activity information.
Concurrency Simulator	Create multiple worker processes to demonstrate contention.

6.5 Input and Output Design

Input	Processing	Output
File name	Validate path and locate managed file	File selected / validation error
Read request	Acquire shared lock and read	File content + operation status
Write request + version	Acquire exclusive lock, validate version, update	New version + success/conflict
Simulation parameters	Spawn worker processes	Concurrent operation log and statistics
Dashboard request	Query SQLite and storage	Current files, metrics, activity

CHAPTER - 7
IMPLEMENTATION

7.1 Module-wise Implementation

The implementation is organized around the principle that all file operations pass through a controlled access layer. The lock is held only for the critical section that requires consistency. Database logging is performed around the operation so that the system can explain what happened during a demonstration.

7.2 Shared Read Lock

```
def read_shared(path):
    with open(path, "r", encoding="utf-8") as f:
        fcntl.flock(f.fileno(), fcntl.LOCK_SH)
        try:
            return f.read()
        finally:
            fcntl.flock(f.fileno(), fcntl.LOCK_UN)
```

The shared lock permits compatible readers to access the file at the same time. The finally block ensures that the lock is released even if an exception occurs.

7.3 Exclusive Write Lock

```
def write_exclusive(path, content):
    with open(path, "r+", encoding="utf-8") as f:
        fcntl.flock(f.fileno(), fcntl.LOCK_EX)
        try:
            f.seek(0)
            f.truncate()
            f.write(content)
            f.flush()
            os.fsync(f.fileno())
        finally:
            fcntl.flock(f.fileno(), fcntl.LOCK_UN)
```

The exclusive lock ensures that the critical write section is not shared with another incompatible operation. The final implementation can combine this lock with the temporary-file atomic replacement strategy described below.

7.4 Version Validation

```
def validate_version(expected_version, current_version):
    if expected_version != current_version:
        raise ValueError("Version conflict: file has changed")
    return True
```

Version validation prevents a client using an outdated copy from silently overwriting a newer update. A successful write increments the version number so subsequent clients can identify the new state.

7.5 Atomic File Update

```
temp_path = target_path.with_suffix(".tmp")
with open(temp_path, "w", encoding="utf-8") as tmp:
    tmp.write(new_content)
```

```
    tmp.flush()
    os.fsync(tmp.fileno())

os.replace(temp_path, target_path)
```

The temporary-file approach ensures that the new content is completely prepared before the original file is replaced. os.replace() provides an atomic replacement operation on the same filesystem.

7.6 Multiprocessing Simulation

```
from multiprocessing import Process

workers = [
    Process(target=reader, args=(shared_file, "P1")),
    Process(target=writer, args=(shared_file, "P2")),
    Process(target=writer, args=(shared_file, "P3")),
    Process(target=reader, args=(shared_file, "P4")),
]

for p in workers:
    p.start()
for p in workers:
    p.join()
```

The simulator starts independent processes that operate on the same shared file. Because the workers use the same file-locking mechanism, the operating system coordinates their access and the dashboard can report successful and blocked operations.

7.7 Database Logging

```
CREATE TABLE operations (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TEXT NOT NULL,
    process_id TEXT NOT NULL,
    filename TEXT NOT NULL,
    operation TEXT NOT NULL,
    status TEXT NOT NULL,
    message TEXT
);
```

The database stores a lightweight audit trail. The exact schema may be expanded with lock type, version, duration, and client information as additional fields.

CHAPTER - 8
EXECUTION, RESULTS AND DISCUSSION

8.1 Execution in Ubuntu

The project is intended to be executed from an Ubuntu terminal. After entering the project directory and preparing the Python environment, the FastAPI application is started with Uvicorn. The dashboard can then be opened in a browser using the local server address.

```
cd SyncLock_Hackathon
python3 -m pip install -r requirements.txt
python3 -m uvicorn app:app --host 127.0.0.1 --port 8000
```

The browser is then opened at http://127.0.0.1:8000. The dashboard provides the controls needed to create a file, select a shared file, read or modify its content, and inspect the activity monitor.

8.2 Demonstration Procedure

10. Open the SyncLock dashboard and show that the system is online.
11. Create or select a shared file such as shared.txt.
12. Show the current version and file contents.
13. Start a read operation and explain the shared lock.
14. Start multiple reader processes and show that compatible reads can proceed.
15. Start two writer processes targeting the same file and show that exclusive locking prevents simultaneous writes.
16. Demonstrate a version conflict by using an old version number for an update.
17. Run the multiprocessing simulator and point out successful operations and blocked/conflicting operations.
18. Show the Activity Monitor and explain how each event is recorded.
19. Conclude by comparing the behavior with an uncontrolled concurrent-write scenario.

8.3 Sample Inputs and Outputs

Scenario	Expected System Behavior	Expected Dashboard Effect
Single read	Shared lock acquired; content returned	Operations +1
Multiple reads	Compatible readers proceed	Operations increase without write conflict
Write with current version	Exclusive lock acquired; write succeeds	Successful Writes +1; version increments
Write while incompatible lock is active	Request waits or is blocked	Conflicts Blocked may increase
Write with stale version	Update rejected	Conflict event recorded
Concurrent simulation	Processes contend for same file	Activity Monitor shows process events

8.4 Results and Discussion

The prototype demonstrates the expected synchronization behavior: read operations can share the resource when compatible, while writes require exclusive access. The use of a real Linux file lock means the demonstration is based on operating-system synchronization rather than only a frontend animation. Version validation adds a second consistency check for stale updates, and operation logging makes the behavior observable.

The dashboard is intentionally designed around four simple indicators: Shared Files, Operations, Conflicts Blocked, and Successful Writes. These values give judges an immediate view of system activity. The activity monitor then provides the detailed event sequence needed to explain why a particular operation succeeded, waited, or was rejected.

8.5 Demonstration of OS Concepts

OS Concept	How SyncLock Demonstrates It
Process Concurrency	Multiple independent worker processes access the same resource.
Critical Section	The protected file update occurs while the required lock is held.
Mutual Exclusion	Only one incompatible writer can hold the exclusive lock.
Reader-Writer Synchronization	Shared reads and exclusive writes use different lock modes.
Race Condition Prevention	Locking controls the order of conflicting operations.
Resource Management	Locks are acquired for an operation and released afterward.
Consistency	Version checks prevent stale updates from overwriting newer data.

CHAPTER - 9
CONCLUSION AND FUTURE SCOPE

9.1 Summary of the Project

SyncLock provides a practical solution for managing concurrent access to shared files. It combines Linux advisory file locking, version validation, atomic file replacement, SQLite logging, a FastAPI backend, and a browser-based dashboard. The multiprocessing simulator makes the synchronization behavior visible by creating real processes that compete for the same file.

9.2 Objectives Achieved

- A controlled shared-file access mechanism was designed and implemented.
- Shared and exclusive locking concepts were incorporated.
- Concurrent process behavior can be demonstrated on Ubuntu.
- Version conflicts can be detected before stale data is committed.
- File operations can be recorded for auditing and visualization.
- The web dashboard provides a simple interface for the hackathon demonstration.

9.3 Results Achieved

The system successfully demonstrates the central objective of preventing incompatible concurrent updates. The dashboard provides a visible representation of operations and conflicts, while the backend performs the actual synchronization. This makes the project both technically meaningful and easy for judges to evaluate.

9.4 Skills and Learning Outcomes

- Understanding of processes, critical sections, mutual exclusion, and race conditions.
- Practical use of Linux file locking and Python's fcntl interface.
- Building REST APIs with FastAPI.
- Working with SQLite for structured application logging.
- Designing a browser dashboard using HTML, CSS, and JavaScript.
- Creating and coordinating independent processes using multiprocessing.
- Testing concurrent behavior and reasoning about consistency.

9.5 Future Scope

- Distributed Lock Manager: extend the design to coordinate processes across multiple machines.
- User Authentication and Roles: support administrators, readers, and writers with controlled permissions.
- Real-Time Updates: use WebSockets or server-sent events to display lock changes instantly.
- Lock Timeouts and Recovery: automatically recover from abandoned locks.
- Advanced Conflict Resolution: provide merge or compare options instead of only rejecting stale updates.
- Scalable Database: move from SQLite to PostgreSQL for larger multi-user deployments.
- Containerized Deployment: package the application with Docker for repeatable deployment.

- Enhanced Analytics: provide charts for lock wait time, contention, success rate, and operation frequency.

CHAPTER - 10
REFERENCES

10.1 Linux / GNU Documentation

20. Linux man-pages Project, “flock(2) — apply or remove an advisory lock on an open file.” https://man7.org/linux/man-pages/man2/flock.2.html
21. Linux man-pages Project, “flock(1) — manage locks from shell scripts.” https://man7.org/linux/man-pages/man1/flock.1.html
22. Python Software Foundation, “fcntl — The fcntl and ioctl system calls.” https://docs.python.org/3/library/fcntl.html

10.2 Framework and Database Documentation

23. FastAPI Documentation, “FastAPI.” https://fastapi.tiangolo.com/
24. SQLite Documentation. https://sqlite.org/docs.html
25. Python Documentation, “multiprocessing — Process-based parallelism.” https://docs.python.org/3/library/multiprocessing.html

10.3 Related File-Locking Systems

26. Git LFS Documentation, “git-lfs-lock — Set a file as locked on the Git LFS server.” https://github.com/git-lfs/git-lfs/blob/main/docs/man/git-lfs-lock.adoc
27. GitLab Documentation, “File locking.” https://docs.gitlab.com/user/project/file_lock/
28. Redis Documentation, “Distributed Locks with Redis.” https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/

10.4 Project Execution Reference

The project is designed for local execution on Ubuntu Linux. The implementation uses Python, FastAPI, Uvicorn, SQLite, HTML/CSS/JavaScript, Linux file locking through fcntl.flock(), and multiprocessing for the concurrency demonstration.

APPENDIX – HACKATHON DEMONSTRATION SCRIPT

A.1 Judge Explanation

"The problem we are solving is concurrent access to a shared file. If several processes write to the same file without synchronization, one process can overwrite another process's update. SyncLock treats the file as a shared resource and controls access using Linux file locks. Multiple readers can use a shared lock, but a writer needs an exclusive lock. We also maintain a version number so that an old copy cannot overwrite a newer copy. The dashboard shows operations and conflicts, and the multiprocessing simulator creates real processes to prove that the mechanism works under contention."

A.2 Recommended Live Demo Sequence

29. Show the dashboard and identify the four counters.
30. Create/select shared.txt.
31. Show its version and content.
32. Run multiple readers.
33. Run competing writers.
34. Show a blocked/conflicting operation.
35. Demonstrate a stale-version rejection.
36. Run the multiprocessing simulator.
37. Open the Activity Monitor and explain the event sequence.
38. Finish by showing that the file remains consistent after concurrent activity.

A.3 Key Terms for Viva

Term	Meaning in SyncLock
Race Condition	The result depends on the timing/order of concurrent operations.
Critical Section	The part of the operation that must be protected from incompatible access.
Mutual Exclusion	Only one incompatible writer can hold the exclusive lock.
Shared Lock	Allows compatible concurrent readers.
Exclusive Lock	Prevents other incompatible operations while a writer updates the file.
Version Conflict	The client version differs from the current server/file version.
Atomic Replacement	A completed temporary file replaces the target as one filesystem operation.
Process	An independently executing program instance used by the simulator.